Comparative study of symbolic execution tools applied to vulnerability detection

André Cardoso
School of Technology
Polytechnic Institute of Cávado and Ave
Barcelos, Portugal
a18848@alunos.ipca.pt

***Abstract*—This document is a model and instructions for LaTeX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

***Index Terms*—component, formatting, style, styling, insert**

I. INTRODUCTION

In the digital era, it is evident that most individuals interact with devices running software that may be susceptible to exploitation. A fundamental principle in cybersecurity is that all software contains vulnerabilities; therefore, its security posture depends primarily on the difficulty of detection. The risk of attack correlates with the diligence of security teams in remediation efforts and the potential incentives available to attackers.

A. Motivation and Background

Considering this, software organizations must take action in defending their codebases in the most efficient manner when it comes to resources and time. Manual validations are based in human interaction, which is known to be very prone to errors, making it not cost-efficient. Also, as an application evolves in complexity, manual validation is not viable when it comes to the resources required, but also time as this is an operation that requires celerity. This procedure is often described as Application Security Testing (AST).

As such, many tools to find vulnerabilities in an automated fashion have been developed and distributed, each with their own set of pros and cons. The tools in question can be categorized as static or dynamic, and each on their own use different technics to accomplish different results. Static Application Security Testing (SAST) ultimately relies on some variety of pattern matching, data-flow analysis, and symbolic execution [1], while Dynamic Application Security Testing (DAST) excels at mimicking realistic attacks and threats [2].

B. Objectives

The objective is a comparison of two symbolic execution tools and see how they compare in terms of performance and their ability to detect vulnerabilities. To achieve this comparison, the tools must run on projects that are preferably coded in the same language, but if these tools do not support the same set of languages, the projects should have similar architecture or business logic/intent (e.g.: online stores, accounting services, ...). These tools will also be compared on how they can be tuned in order to achieve better results.

For a better overview of how the comparison will be performed, here follows a set of questions and metrics used to measure the tools:

- Time to test a project
- CPU usage while testing a project
- Memory usage while testing a project
- Analysis of vulnerabilities detection (Youden's Index)
- Complexity of the vulnerabilities detected
- Ability to find vulnerabilities out-of-the-box
- Ability to find vulnerabilities with proper manipulation
- Effort of tuning the tool

In sum, the expected output is a clear comparison of tools that can assist developers and software companies in keeping code secure proactively and explain what differs between them and why one should pick one over the other(s). These is how we intend to answer the following research questions:

- **Which tool is more effective at finding vulnerabilities?**
- **Which tool is more efficient in terms of performance?**

C. Research Method

Design-Science Research is the most suitable research method for this work as it focuses on the creation and evaluation of artifacts that solve identified organizational problems [3]. In this case, the organizational problem is the need for efficient and effective tools to identify vulnerabilities in software applications, which is a critical concern for software development organizations aiming to enhance their security posture.

[3] outlines 7 guidelines for conducting Design-Science Research, which will be followed in this study:

1) **Design as an Artifact:** The primary artifacts in this research is the research itself - a comparative analysis of symbolic execution tools for vulnerability detection - and the repository with the samples used in the experiments (see section IV).
2) **Problem Relevance:** The research addresses the practical problem of identifying effective symbolic execution tools for vulnerability detection in software applications. This research also guides practitioners in the setup and usage of these tools (see section III).

3) **Design Evaluation:** The evaluation of the symbolic execution tools will be conducted through empirical testing on selected software projects, measuring performance metrics and vulnerability detection capabilities (see section IV).
4) **Research Contributions:** The study aims to contribute to the field of software security by providing insights into the effectiveness and efficiency of different symbolic execution tools and offering guidance for practitioners in selecting appropriate tools (see section IV, particularly section IV-K).
5) **Research Rigor:** The research will employ rigorous methodologies for tool selection, testing, and data analysis to ensure the validity and reliability of the findings (see section IV-A).
6) **Design as a Search Process:** The research will involve an iterative process of tool selection, testing, and refinement to identify the most effective and/or efficient symbolic execution tools (sections II-D1, III-A and IV).
7) **Communication of Research:** The findings of the research will be documented and disseminated through academic publications and presentations to reach both the academic community and industry practitioners. This document serves as the primary medium for communicating the research process and findings, and the testing is shared in a public repository (url: https://github.com/18848/masters_thesis).

The comparison of symbolic execution tools can be summed up by the question "what is effective" that [3] explained when discussing the behavioral-science and design-science paradigms. The question proposed to answer looks into the design-science paradigm has it is required an evaluation of the available technologies, "Design as a Search Process".

To perform a valid comparison between symbolic execution tools, firstly there must be a selected set of tools to be compared and for such there must be set requirements that an engine has to fulfil in order to be elected as target of comparison. These requirements have to ensure the validity of the comparison of different tools, there are no two equal pieces of software, but they have to share some characteristics in order for a comparison to be logic.

To compare these tools, execution and analysis must be done. For this to be properly done, one has to understand how and why they work, which requires a more theoretical study of these tools and consequently to research papers on and about them as well as documentation.

As symbolic execution engines in their nature require source code of some kind, we will need to research and select some open-source software projects that allows the comparison of different tools. Fitting examples of this are the Test-Comp benchmarks [4]. This is where the previous step of setting requirements proves necessary; the more similar the different tools are, the more likely they are to be able to run on one same project, which allows for a better test case.

Once prerequisites are fulfilled, selecting tools, setting them up and finding projects, the experimental phase begins; first the trial is done without any sort of modifications to understand the baseline of each tool:

- Is the tool useless without proper tuning?
- Can it find simple vulnerabilities out of the box?
- Where/When does it become harder to find vulnerabilities?
- How well does it perform in time and space?
- How well does it perform under the Youden's Index?

After answering these questions and recording the results, the investigation moves to the modifications phase:

- How can the tool be improved/tuned?
- How expensive is it to improve the results? (resources spent tuning the tool)
- Can we improve the tool in the number of results? How?
- Can we improve the tool in terms of performance? How?
- Can we improve both metrics at the same time?

If we're able to answer the previous questions, what follows is a comparison with the first set of results

- Did we find more vulnerabilities?
- How complex are the found vulnerabilities?
- How does the Youden's Index compare?
- How does the cost of working on the tool compare to the results' improvement?

All the previous points will be recorded, analyzed and compared in order to achieve a final comparison of the selected symbolic execution tools. The definition of these steps and questions is crucial to ensure that the comparison is fair, thorough, and provides valuable insights into the capabilities and limitations of each tool, as determined in the Research Rigor guideline.

D. Document Structure

Section II, LITERATURE REVIEW, presents a brief introduction to the theme at hand and its relevance to today's software dilemma of Application Security Testing. Two relevant methodologies of application testing are also discussed: SAST and DAST.

To start the comparative work, section III, SYMBOLIC EXECUTION TOOLS, introduces the tools that will be tested, compared, and discussed in terms off what they offer, how they work and how to interact with them. For each tool, multiple topics will be covered: installation, available features, source code instrumentation, and finally, how to run the tools in the instrumented sources.

Once everything required to start with section IV, EXPERIENCES AND RESULTS, is ready, the code samples (or test cases) are introduced, the tests are conducted, and the information is collected. Before addressing the samples and testing, the test environment is described in detail, from how each sample's working directory was prepared to the testbed specifications. Only then is each sample detailed, instrumented, and scanned by both tools. In the end, multiple comparisons are done, and results are presented, showcasing performances and features.

Finally, the **??** states how each tool compares, their weaknesses and strengths, as well as future work and possible improvements on this comparative study.

II. LITERATURE REVIEW

The security of an application showed its first impacts around the beginning of the century with the increasing popularity of the World Wide Web (WWW) and the enablement of user-fuelled websites (like social media). This rapid evolution paired with the common lack of knowledge of the users but also of the developers caused the birth of Denial of Service and Cross-Site Scripting attacks among others [5].

A. Application Security Testing

Application Security Testing (AST) includes every procedure that evaluates software security, these methods may include manual or automatic testing; manual testing may perform in the form of penetration tests, but automatic testing has 2 common variants, static (SAST) and dynamic (DAST).

Static Application Security Testing is a white-box approach to software security where we have access to source code and through analysis a developer can attempt to find vulnerabilities in it; its practice is also commonly described as finding code errors.

Dynamic Application Security Testing is on the other hand also known as the black-box approach – it has no knowledge of how a software is coded or designed. It only knows how to interact with the software and attempts to do so in a manner that it can find malfunctions and failures in the program execution; its results are the inputs used to break the application flow.

There is also a practice known as Interactive Application Security Testing which despite being described as a white-box technique, it also examines the software security during runtime, and it must be integrated with it. However, this represents runtime overhead and inability to detect client-side vulnerabilities [6].

1) SAST vs. DAST: After looking at the research performed by [7], a summarized version of the comparison between the two methods can be described as seen in Table I.

SAST	DAST
White-box testing	Black-box testing
Test from the inside out	Test from the outside in
Test internal behaviour	Test external expectations
Validate code quality	Validate system functionality
High granularity	Low granularity
Full knowledge of application structure	No knowledge of application structure

TABLE I: Summarized Theoretical Comparison of SAST & DAST

This theoretical comparison is presented with a more practical one, where [7] tests, analyses, and studies different scans with different tools for both methods. The metrics used are the standard True Positive, True Negative, False Positive, False Negative (TP, TN, FP, FN) and are required to calculate the True Positive Rate (TPR) and False Positive Rate (FPR) which happen to be the most relevant rates that contribute to a grade of accuracy and confidence in each cybersecurity tool used. One such way to infer an accuracy metric is using Youden's Index that can be seen in equation (1) as **(author?)** [8] described for the OWASP Benchmark.

Youden's Index or Youden's J statistic

$$Youdenindex(Score) = TPR(TruePositiveRate) - FPR(FalsePosit \quad (1)$$

(author?) [9] also describes DAST as "especially helpful for detecting: input or output validation, authentication issues, server configuration mistakes" and specifies that they "allow for sophisticated scans on the client side and server side" and require no implementation information (source code, framework, etc.).

2) Software Development Life Cycle: An analysis of software security necessarily encompasses the entire development lifecycle, with particular emphasis on testing and deployment. Integrating specialized security testing within the validation phase is essential; however, the deployment phase remains equally critical, as it represents a high-risk transition point within the system's lifecycle [10]. Moreover, it is worth noting how the cost of remediating a risk increases the further down the life cycle it is detected and flagged when compared to those found earlier [11].

After a careful analysis of the two techniques to test application security, and a quick glance at Figure 1, what makes most sense for most organizations is to execute static security scans as part of the testing phase. If a more complete deployment is required for dynamic tests, then we could be looking at a later stage: deployment. This also depends greatly on how companies implement CI/CD; if they do it well enough, then the testing phase just became much more agile and dynamic, making it possible to secure software statically directly in the implementation phase and dynamically in the testing phase, detecting problems much earlier and reducing costs in overall risk management [7, 12].

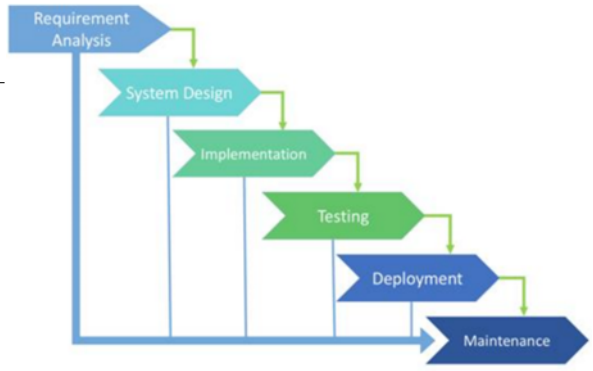

Fig. 1: Software Development Life Cycle Model [10]

B. SAST - Static Application Security Testing

Expanding on the white-box technique, most vulnerabilities are due to developer's lack of awareness and/or bad pro-

gramming practices [11] in such a way that **(author?)** [13] defines it "as part of a Code Review", a "method of computer program debugging that is done by examining the code without executing the program", and also a good technic to help find:

- "Syntax violations"
- "Security vulnerabilities"
- "Programming errors"
- "Coding standard violations"
- "Undefined values"

1) Symbolic Execution: Inside SAST's set of technics, introduced by [14], we find symbolic execution engines which can help protect systems that range from modern System-on-Chip (SoC) designs and hardware Trojans [15], to binary-level security analysis such as malware comprehension [16] and even a symbiosis between symbolic and concrete execution [17]. All these tools share a common requirement: any instruction set ranging between machine code to source code. These tools excel by mimicking the execution of the analysed software, controlling each value the input influences in the form of symbolic values or variables.

However, a more in-depth look at symbolic execution unveils some obstacles, such as path explosion, constraint solving, environment modelling and others. These symbolic values are allowed to be anything at a first stage, and its value is deciphered further down in the execution path, reducing the possibilities at every branch the execution encounters. The branching happens after performing constraint solving, a very time expensive operation especially when the tested source is of high complexity [18].

Constraint solving occurs at every moment where a variable value impacts the flow of execution or even once a symbolic variable takes part in any arithmetic computation. This escalates once these computations become of a non-linear nature, such as non-linear expressions and non-linear floating-point computations [19].

Path explosion is one concern found in many high performance problems: the input is sufficiently complex for any sort of resource - be it time, memory, or even processing capacity - to escalate tremendously and becoming unreasonable in a standard environment. When it comes to symbolic execution, a new path is encountered every time a constraint is solved; from that point on, that constraint is a path condition. To better understand this concept, here's an example using the tool *KLEE* according to [20, p. 3]:

> "When KLEE detects an error or when a path reaches an exit call, KLEE solves the current path's constraints (called its path condition) to produce a test case that will follow the same path when rerun on an unmodified version of the checked program (e.g., compiled with gcc)."

Environment modelling is another very common problem in software analysis techniques given how much software development relies on third-party libraries today. This problem escalates whenever these dependencies are packaged as binaries, or the sources are 'super-optimized', the paths exploration and consequently the constraint solving may fail which may result in false positives or false negatives [20].

C. DAST - Dynamic Application Security Testing

The black-box method of testing is very tightly coupled with a hacker perspective of the application; the intention is to find design and/or implementation flaws that cause the system to break down or allow its security to be compromised. Sometimes, to be able to do this a better understanding of the code is required and the testers will frequently find themselves trying to understand how the application is coded. This technique is known for its low false positive rates, its language independent nature and its ability to detect configuration related attacks. The abstraction of the software implementation is possible because DAST tools require a runtime test case, this means that in the SDLC this is applied in the later stages (shift-right) [2].

1) Fuzzing: One way to dynamically test software is through fuzzing techniques; this consists of feeding random input with the objective of breaking the application logic or even its execution. [21] also describes it as "test generation and execution with the goal of finding security vulnerabilities". Although *fuzzing* is a quick technique, it generally has poor path coverage on its own.

2) Dynamic Taint Analysis: [22] explain that dynamic taint analysis also operates in running programs, but this time by carefully analysing application inputs and each operation to keep track of new and derived taint in an attempt to prevent execution of potentially harmful instructions. Usually, a tainted variable is influenced by user input and then every other variable that is influenced by the first. This technique can be summarized in three steps or *taint policies*: taint introduction, propagation, and checking.

All variables are considered untainted and that only changes when raw user input is stored in one of them, then it becomes a tainted variable. Following user input comes application logic, the same logic that through expressions will contaminate any other variable that interacts with the first set that was tainted by user input. However, once the application gets to a critical operation, it can decide on performing it or not based on how tainted critical variables are (taint checking).

D. Symbolic Execution

This work is focused on the static method of application security testing, symbolic execution, and as such we will analyse some available tools. One rather large collection of tools, papers, courses and videos on this thematic is the GitHub repository ksluckow/awesome-symbolic-execution[23].

1) Symbolic Execution Engines: This chapter goes through a selection of engines that were considered for this study: Symbolic PathFinder, Jalangi2, SymJS, Cloud9, Kite, KLEE, and Owi. These aren't necessarily better or worse then others that aren't explicitly mentioned. The selection was focused on two characteristics: wide language support and impact on Web Applications.

Cloud9, Kite, and KLEE, are tools designed, directly or indirectly, for LLVM which supports many different languages [24]. Owi is in itself a toolchain designed to improve the WASM developer experience that also packages a symbolic interpreter for WebAssembly.

Symbolic PathFinder (SPF)

Symbolic PathFinder is an extension for the Java PathFinder (JPF), a 'model checking framework for Java™ bytecode programs' [25, 26]. SPF allows symbolic execution on methods with arguments of basic types, strings, arrays and data structures [25].

Jalangi2

Jalangi2 is a framework for analysis of JavaScript programs. Jalangi2 was designed to work on browser applications and supports analysis that track 'NaNs' or even a memory-profiler but also allows customization through implementation of custom analysis [27].

SymJS

To test JavaScript web applications there is also SymJS, packed with a symbolic execution engine and also 'an automatic event explorer for Web pages'. It is also possible to produce test cases and improve execution through dynamic feedbacks [28]. SymJS references were only found in articles without distributions of software of any kind.

Cloud9

Cloud9 was designed with the intent of reducing the 'resource-intensive ... nature of high-quality software testing', and claims to be 'the first symbolic execution engine that scales to large clusters of machines' [29]. Unfortunately, no source code or executable of this tool was found, the only available link on their homepage wasn't valid.

Kite

Kite is a Proof-of-Concept tool that tackles the symbolic execution problem using the Conflict-Driven Symbolic Execution (CDSE) algorithm which is able to dynamically reduce path exploration and is also capable of learning from earlier conflicts [30]. Similarly to Cloud9, no source or executable was found and the available links were invalid [31].

KLEE

KLEE is perhaps the most relevant tool to address in the LLVM field considering how both Kite and Cloud9 were developed on top of the work acomplished by KLEE. Contrary to what we've seen in the other two tools, KLEE counts with a large community and many papers discussing this tool, its capabilities and other work around it. This is one of our selections for this comparison and more information on it will be presented in section III-B.

Owi

Owi seems to be the only tool explicitly focused on the WebAssembly binary format in the list aggregated by **(author?)** [23]. Nevertheless, it presents promising work specially by their attempt to integrate with the WASM developer lifecycle with subcommands that allow for other features useful for the everyday tasks like formatting of files and validation of modules [32]. These and many more features including symbolic execution will be discussed in section III-C as this is the second choice for comparison.

III. SYMBOLIC EXECUTION TOOLS

There are a couple of tools available that perform symbolic execution in numerous ways with different focus and strengths. A good agglomerate of these tools is maintained in a GitHub repository owned by **(author?)** [23]. There we find symbolic execution tools focused on binaries and various programming languages like Java, JavaScript, or even Rust, but also portable platforms like WASM and LLVM.

This chapter focuses on the tools that were considered for this work, and the reasons behind the final choice (section III-A). Then, the chosen tools, KLEE and Owi, are described in more detail. For both tools, installation instructions, features, and usage examples are provided (sections III-B and III-C, respectively). Finally, a comparison between both tools is presented in section III-D.

A. Engines Availability

In the endeavour of searching for tools capable of symbolic execution many attempts to have a functional installation of the available alternatives were done, but not all were successfull.

Despite of Cloud9 and Kite having their own websites [29, 31] both did not contain valid hyperlinks for either sources or a compiled binary of the software. On the other hand, for SymJS nothing more than a scientific document was found [28].

That leaves us to KLEE and Owi, a pair of tools with a large support of languages [24, 33], and SPF and Jalangi2, the two programming languages used to develop the WebGoat project maintained by OWASP [34, 35]. OWASP WebGoat project is an intentionally vulnerable application so developers interested in learning cybersecurity can use it to learn about vulnerabilities [35].

Despite the interest around OWASP's WebGoat project and the availability of sources for both SPF and Jalangi2, getting each software to run on their own and consistently is a daunting task with not much documentation to go along and a steep learning curve.

KLEE and Owi are both far better when it comes to the available documentation, and initial learning curve; the users are not bombarded with a codebase and a single 'README' file, instead they have either multiple README's for each subcommand [32] or a large number of papers and websites with tutorials, examples and information on the tool features [20, 36–39].

Given the current software paradigm, where performance is important not only in the matter of software runtime but

also when it comes to software development, maintenance, and portability, tools that apply to technologies such as LLVM and WASM are interesting research targets. Therefore, we are looking at KLEE and Owi, which propose solutions for each tech stack, respectively.

B. KLEE

KLEE explores the LLVM infrastructure rather successfully and with speed; however, this engine bottlenecks quite considerably when it is applied on larger scale projects. This is due to its space management, given how it tries to keep all relevant information about the current states. It is also unable to handle dynamically generated code, concurrent execution and struggles with modelling network and peripherals, although most peers share these weaknesses [40]. The LLVM project supports many different languages like Haskell, Rust and Python, but our test cases will focus on C/C++ programs [24]. .

LLVM has been referenced a few times in this document, but a brief explanation is in order. LLVM is not an acronym, but the full name of the project. Despite the name, it has nothing to do with virtual machines. LLVM is "a collection of modular and reusable compiler and toolchain technologies". It is also praised for its versatility, flexibility, and reusability. For more information about LLVM, please refer to their website [24].

[20] describe KLEE as being in both static and dynamic techniques of testing: every symbolic execution engine is static by definition, but KLEE becomes dynamic through its outside environment interaction features.

1) Installation: The KLEE CLI tool is available in many ways and one of the fastest ways to get started is through their Docker image, but they also publish to the Snap Store and many package managers. However, the KLEE authors recommend building from source.

A manual installation is demanding immediately from the dependencies' installation - 16 packages are required to install KLEE, 2 of which are optional (to generate documentation), and 3 more are recommended for improved usability. All the required commands are available in the **??** 1.

```
# graphviz/doxygen are optional (documentation generation)
sudo apt-get install build-essential cmake curl file \
    g++-multilib gcc-multilib git libcap-dev \
    libgoogle-perftools-dev libncurses5-dev libsqlite3-dev \
    libtcmalloc-minimal4 python3-pip unzip graphviz doxygen

# additional features in 'klee-stats'
sudo apt-get install python3-tabulate

# 'lit' enables testing
# 'wllvm' makes it easier to compile to LLVM bitcode
sudo apt-get install pipx
pipx install lit wllvm
```

Listing 1: KLEE dependencies installation.

Base dependencies aside, KLEE still requires critical tools such as LLVM 13 and at least one constraint solver; KLEE was built on top of LLVM 13 and the STP solver tools, but also supports Z3. LLVM 13 and the STP solver can be installed by using the commands in **??** 2–4.

```
# add LLVM repositories to 'apt' package manager
wget -O - https://apt.llvm.org/llvm-snapshot.gpg.key
    ↪  | sudo apt-key add -

# install LLVM packages
sudo apt-get install clang-13 llvm-13 llvm-13-dev
    ↪ llvm-13-tools
```

Listing 2: LLVM installation.

```
# STP dependencies
sudo apt-get install cmake bison flex libboost-all-
    ↪ dev python perl zlib1g-dev minisat

# clone MiniSAT
git clone https://github.com/stp/minisat.git
cd minisat

# build
mkdir build
cd build
cmake -DCMAKE_INSTALL_PREFIX=/usr/local/ ../

# install
sudo make install
```

Listing 3: STP dependencies installation.

```
# clone STP
git clone https://github.com/stp/stp.git
cd stp
git checkout tags/2.3.3

# build
mkdir build
cd build
cmake ..
make

# install
sudo make install

# increase stack size to run KLEE on larger benchmarks
ulimit -s unlimited
```

Listing 4: STP installati

2) Features: KLEE is packaged with multiple CLIs (see Table II) that contribute to the user experience, but the main command that runs the symbolic execution engine is `klee`.

Command	Description
`kleaver`	Operates on `.kquery` files.
`klee-exec-tree`	Takes `klee` tree output and displays a graphical representation o statistics.
`klee-replay`	Takes one ktest from a file or multiple from a directory and runs
`klee-stats`	Takes `klee` stats output and processes it as instructed.
`klee-zesti`	ZESTI[1] like wrapper of KLEE.
`ktest-gen`	Generates test cases (a `.ktest` file) from concrete input.
`ktest-randgen`	Generates test cases (a `.ktest` file) from randomly generated in
`ktest-tool`	Describes a `.ktest` file containing information on each input ob

TABLE II: KLEE commands.

3) Instrumentation: The KLEE web page contains a tutorials section to help get familiarized with the tool. These tutorials contain samples and instructions on how to properly

set up a C/C++ source file, let's call it code instrumentation, and how to run and analyse the results. Their sources can be found in higher detail in appendix **??**.

The first, and most basic example, shows how one can mark a variable as symbolic by using the `klee_make_symbolic()` function (see **??** 6). This function takes 3 arguments; the first two will define a memory range in which the variable information is stored, the third defines a human-readable object name that identifies the symbolic variable in the generated test case files (for the variable '`a`' declared on section III-B3 in **??** 6, this could be "my variable".):

1) The variable memory address,
2) The variable memory size,
3) The variable name, a human-readable identifier.

Running `klee` on the compiled LLVM bitcode did not return any error but does return a couple of other interesting details. On section III-B3 of the **??** 5, we have information on the number of completely and partially complete traced paths, and lastly the number of generated test cases. Every traced path has a corresponding test, but only detected errors generate error information and an error query; these two different informations will be covered later once we start looking at test examples on section IV EXPERIENCES AND RESULTS.

```
KLEE: Using STP solver backend
KLEE: SAT solver: MiniSat
KLEE: Deterministic allocator: Using quarantine queue size 8
KLEE: Deterministic allocator: globals (start-address=0x7fe2cf21c000 size=10 GiB)
KLEE: Deterministic allocator: constants (start-address=0x7fe04f21c000 size=10 GiB)
KLEE: Deterministic allocator: heap (start-address=0x7ee04f21c000 size=1024 GiB)
KLEE: Deterministic allocator: stack (start-address=0x7ec04f21c000 size=128 GiB)

KLEE: done: total instructions = 12170
KLEE: done: completed paths = 3
KLEE: done: partially completed paths = 0
KLEE: done: generated tests = 3
```

Listing 5: Tutorial 1 `klee` output

```
// including klee library is optional
//#include <klee/klee.h>

int get_sign(int x) {
  if (x == 0)
    return 0;

  if (x < 0)
    return -1;
  else
    return 1;
}

int main() {
  int a;
  klee_make_symbolic(&a, sizeof(a), "a");
  return get_sign(a);
}
```

Listing 6: Tutorial 1 sample

Another valuable feature, is the `klee_prefer_cex()` that tells KLEE to prefer a set of values when generating test cases; this allows the user to get readable results like "aaaa" instead of hexadecimal representation of character values such as "\xff\xff\xff\xff".

4) Running KLEE: After instrumenting the source code with KLEE instructions, see section III-B3, the next step is compiling it to LLVM, and the easiest way to achieve that is using Clang (**??** 7).

```
clang -emit-llvm -c main.c
```

Listing 7: Basic clang command

When compiling programs that include the klee library, it may be needed to include the klee library, so Clang understands its instructions, which can be done like in **??** 8.

```
clang -I $PATH_TO_KLEE_LIBRARY -emit-llvm -c main.c
```

Listing 8: Basic clang command with path to klee library

However, the KLEE documentation recommends a more complete command to have access to a more complete set of functionality that klee provides, such as test replayability, and that command is available in **??** 9.

```
clang -emit-llvm -c -g -O0 -Xclang -disable-O0-optnone main.c
```

Listing 9: Complete clang command

The compilation command should return an object with extension '*.bc*' containing the LLVM representation of the program. This file is in the correct format for KLEE, and therefore we can now test it by running the command on **??** 10.

```
klee main.bc
```

Listing 10: Running klee on a program

Running KLEE on a program results in file outputs, the `klee-out-n` and `klee-last` folders are created. For `klee-out-n`, `n` is the zero-indexed counter of the run, and `klee-last` is a symbolic link to the most recent run folder; by the second run of KLEE, `klee-last` points to `klee-out-1`. The folders created by KLEE contain all the generated test cases, error reports, and other useful information about the execution. Each generated test case is stored in a file with extension '*.ktest*' and can be analysed using the `ktest-tool` command (see **??** 11).

```
$ ktest-tool klee-last/test000001.ktest
ktest file : 'klee/klee-last/test000001.ktest'
args       : ['main.bc']
num objects: 1
object 0: name: 'my signed integer'
object 0: size: 4
object 0: data: b'\x00\x00\x00\x00'
object 0: hex : 0x00000000
object 0: int : 0
object 0: uint: 0
object 0: text: ....
```

Listing 11: Analyzing a ktest file

C. Owi

[4] describe Owi as a toolkit that aggregates functionality to assist WebAssembly developers, one of which enables the compilation of C code to WASM and running their symbolic interpreter on top of it.

WebAssembly is a portable binary instruction format for executable programs, designed to be a compilation target for high-level languages like C/C++ and Rust, enabling deployment on the web for client and server applications. WebAssembly is designed to be fast, efficient, and secure, making it an ideal choice for running complex applications in web browsers and other environments [41, 42].

1) Installation: To properly make use of the tool, a working installation is required and Owi is officially available in two ways. The first and more direct approach is by using the opam, the OCaml Package Manager. The second alternative is through a manual installation using *dune*, the OCaml build system, which can also be installed using *opam* or manually using *make* commands.

The first method resulted in an unusable binary (when it comes to symbolic execution), without any helpful information or help page when running it, therefore is recommendable the use of the second method and building the tool from the repository yourself. To do so, all that needs to be done is running the instructions in **??** 12 to install `opam`, the required dependencies, which include `dune`, and finally building and installing `owi`.

```
# install opam
apt install opam
# install owi dependencies
opam install . --deps-only
# build owi
dune build -p owi @install
# install owi
dune install
```

Listing 12: Owi installation

2) Features: Owi being a toolchain designed for the development of WASM, it provides a set of subcommands as listed in Table III; however, to be able to use the symbolic interpreter packaged with it, we need to also install a restraint solver. Owi supports any Smt.ml supported solver, but the default solver is Z3.

Owi also supports test replayability from an output file generated by the *sym* subcommand as you can see in **??** 13.

```
owi sym main.wat > main.model
owi replay --replay-file=main.model main.wat
```

Listing 13: Owi replayability model

3) Instrumentation: The Owi repository maintains a folder with examples of how to use the different subcommands; similarly to KLEE, we have available multiple examples on how to use `owi c` that are detailed in appendix **??**.

Looking at the first example, some differences from KLEE are evident, specifically how the tool is informed about which variables are symbolic. KLEE uses the variable address and the type is disregarded in the whole process (the generated test cases detail values for each possible primitive type); Owi on the other hand expects to be told the symbolic variable type, through the available functions:

Subcommand	Description
c	Compiles C source code to WASM and runs the WASM symbolic interpreter on the end result.
conc	Takes WASM input and runs the concolic interpreter on it.
fmt	Formats WebAssembly text format files (*.wat* or *.wast*).
opt	Optimizes WebAssembly modules.
run	Runs the concrete interpreter.
replay	Replay a WASM module containing symbols with concrete values in a replay file containing a model.
script	Runs a reference test suite script.
sym	Takes WASM input and runs the symbolic interpreter on it.
validate	Validates WebAssembly modules.
wasm2wat	Generate a text format file (*.wat*) from a binary format file (*.wasm*).
wat2wasm	Generate a binary format file (*.wasm*) from a text format file (*.wat*).

TABLE III: Owi subcommands

- `owi_i32()` - to define 32-bit integer values, with 8-bit and 64-bit
- `owi_f32()` - to define 32-bit floating-point values, with a 64-bit v
- `owi_char()` - to define character values;
- `owi_bool()` - to define boolean values.

Unlike KLEE functions, these do not support any argument that allows defining a custom name for the symbolic variable; the symbols are identified by order of creation - if we define all positions of an array as symbolic, each position symbolic identifier would correspond to their array position, but if any other symbol is defined before the identifiers would shift by the number of predefined variables.

To run Owi on a simple sample we must include the Owi library in the source code so it is possible to find Owi function implementations and generate a valid WebAssembly module. As visible on **??** 14, the variable `x` of type `int` is instrumented using the routine `owi_i32` (section III-C3) and since this program is trying to find the roots of a polynomial, we want the symbolic values that result in `poly == 0`, therefore section III-C3 will catch a 'problem' every time `poly` equals 0 since the passed condition evaluates to `false`.

There are in fact solutions to the defined polynomial and Owi outputs them as visible in **??** 15. The output represents a test case, and to replicate the test case one or more of the passed inputs must have a specific set of values. On the example at hand, only one symbol was defined and its state is displayed on section III-C3. The second value on this line is the symbols' name (`symbol_0`), then follows the symbol's type and since an integer was defined its type `i32`. Finally, its value is also presented, in this case we can state that 4 is one solution to the polynomial written on **??** 14.

Owi also produces a `owi-out/test-suite` folder and inside it a `testcase-N.xml`, where `N` is an index starting on 1, that describes the input value for a symbolic variable for that test case, and a `metadata.xml` that details the arguments of the owi executation such as the source code language, which Owi subcommand produced the file, the input program file ('*poly.c*') and other details on the execution. The generated test cases are relevant to found issues, Owi does not generate

test cases on each traced path like KLEE does.

```c
// this include is required
#include <owi.h>

int main() {
  int x = owi_i32();
  int x2 = x * x;
  int x3 = x * x * x;

  int a = 1;
  int b = -7;
  int c = 14;
  int d = -8;

  int poly = a * x3 + b * x2 + c * x + d;

  owi_assert(poly != 0);

  return 0;
}
```

Listing 14: Polynomial example 1 source code with Owi instrumentation

```
Assert failure: (bool.ne
                  (i32.mul
                   (i32.add (i32.mul (i32.add
    ↪ symbol_0 -7) symbol_0) 14)
                   symbol_0) 8)
model {
  symbol symbol_0 i32 4
}
Reached problem!
```

Listing 15: Polynomial example 1 output

4) Running Owi: Similarly to KLEE, Owi provides a set of methods to inform the symbolic interpreter in an attempt to improve the results, examples of this are `owi_assume()` and `owi_i32()` for int32 variables (and others for the respective primitive types such as `owi_char()`).

Once the sources are ready, running the symbolic interpreter is very simple with Owi by running Listing 16. Since this is a toolkit it contains many subcommands, the most relevant for this study being the '*c*' subcommand that allows running C code directly.

```
owi c main.c
```

Listing 16: Running owi on a program source.

However, if your program requires any sort of custom option or if you just want to experiment with them, we can 'ask' Owi how they compile the sources. To do that, all one needs to do is run `owi c main.c --debug` on invalid C files and will find in the tool output the command in **??** 17.

```
clang -O3 --target=wasm32 -m32 -ffreestanding \
    ↪ --no-standard-libraries -Wno-everything -flto
    =thin \
    ↪ -Wl,--entry=main -Wl,--export=main -Wl,--lto-
    O0 \
    ↪ -Wl,-z,stack-size=8388608 \
    ↪ -I $HOME/.opam/default/share/owi/libc -o main
    .wasm \
    ↪ $HOME/.opam/default/share/owi/binc/libc.wasm
    \
    ↪ main.c
```

Listing 17: Owi abstracted compilation command (clang).

Owi also integrates Frama-C's E-ACSL plugin. This plugin enables the detection of implementation issues, or even custom heuristics for complex bugs, through the definition of a set of rules that can create constraints on acceptable inputs ('requires') and outputs ('ensures'). The `owi c` subcommand and to enable all one needs to do is pass the argument `--e-acsl`. To see how Frama-C is being executed the process is similar as to the compilation, but now with the E-ACSL option, and it looks like **??** 18. This plugin enables the detection of implementation issues, or even custom heuristics for complex bugs, through the definition of a set of rules that can create constraints on acceptable inputs ('requires') and outputs ('ensures').

```
frama-c -e-acsl -no-frama-c-stdlib \
    ↪ -kernel-warn-key CERT:MSC:38=inactive -
    verbose 2 \
    ↪ -cpp-extra-args="-I$HOME/.opam/default/share/
    owi/libc" \
    ↪ main.c -then-last -print -ocode main.
    instrumented.c
```

Listing 18: Owi abstracted Frama-C instrumentation of E-ACSL.

After compiling either `main.c` or `main.instrumented.c`, the resulting `.wasm` object can be tested with Owi using the command on **??** 19.

```
owi sym main.wasm
```

Listing 19: Running owi on a compiled program.

Executing either `c` or `sym` subcommands generates some files inside the folder 'owi-out' created on the current working directory: `owi sym` only generates test case files named 'testcase-n.xml' being n the number of results detected and starting from 1. The subcommand `owi c` also generates a file describing the test cases that fail execution, but also outputs an additional 'metadata.xml' as described in Table IV, and a 'a.out.wasm' file (the compiled sources); however, the test case and metadata files are generated in a subfolder called 'test-suite' while the WebAssembly module is created on the current working directory (at the same level as 'owi-out' folder). For a visual reference of this file structure take a look at **??** 20, consider 'main.c' the input sources and 'main.wasm' pre-compiled before running Owi.

Field	Example	Description
sourcecodelang	'C'	The programming language in which the input sources are written
producer	'owic'	The subcommand responsible for this metadata file.
specification	N/A	No information on this field was found
programfile	'main.c'	The input source files
programhash	b663247...	The compiled wasm module hash
entryfunction	'main'	WASM modules requires by default a specific entry point/function, in this case Owi defines 'main'
architecture	'32bit'	The target architecture (defined by Owi)
creationtime	timestamp	The day and time this metadata file was created

TABLE IV: Owi 'metadata.xml' file contents.

```
# owi c ...
.
├── a.out.wasm
├── main.c
└── owi-out
    └── test-suite
        ├── metadata.xml
        └── testcase-1.xml

# owi sym ...
.
├── main.c
├── main.wasm
└── owi-out
    └── testcase-1.xml
```

Listing 20: Owi generated files structure

D. Feature Comparison

Not only it is important to understand how well a tool performs, but also how flexible it can be in the different scenarios it may face. Table V was built describing each tool supported vulnerabilities, their ease-of-use, how easy they are to install, and which STP solvers they support. The tools' installation speed and complexity are scaled from 1-5 ranging from very slow to very fast or very hard to very easy, respectively. The third and fourth columns (Ease-of-use for beginner and experienced developers) take the same scale as the second one: 1-5, very hard to very easy.

The tools' installation process is very different both in the amount of methods but also in the complexity of the provided solutions. Although the recomended solution by **(author?)** [39] is very complex since it requires a lot of dependencies and different processes, as we discussed in section III-B1, there is also available a quick start option using a docker image made available by the maintainers of KLEE. In contrast, Owi does not offer many ways to install its engine but both of them require the installation of `opam` but also `dune` if one compiles the source code directly.

With their differences in mind, KLEE punctuates a score of 5 in installation speed but if we disregard the Docker approach, both KLEE and Owi take 2 points; even though the first requires much more commands and dependencies, the second takes more time to actually compile the repository.

When it comes to installation complexity, KLEE clearly could use some improvement in build automatization while Owi leverages this quite well with `opam` and `dune` however these are tools very tied with OCaml development which is not at all the focus of this study adding unnecessary complexity to the installation process. Considering this, KLEE scores a 1 in the recommended process. But if using Docker is an option, then it scores a 4. Owi on the other hand scores a 3, much because of the OCaml tools; we're intentionally ignoring the opam package since it did not produce a valid installation, otherwise Owi could score a 5.

The tools usability is also scored in the perspective of both beginners and experienced developers. Because KLEE has so much documentation available across the internet the learning curve is very stable but the more advanced the requirements the more complex the usage may get due to how some functionalities are scattered across different CLI's, these two points contribute for a solid positive score (4). Owi on the other hand can pose as a hard obstacle for the least experienced mainly due to the lack of support documents and the fact that OCaml doesn't make it into the top 50 of most popular programming languages where the 50th most popular language rates 0.15%, according to **(author?)**[43], scoring 2 points in this scenario. The popularity issues become nearly non-existent when an experienced mind makes use of the tool and the simplicity of having all tools aggregated into one CLI also contributes to a better score (4).

The second section of Table V describes features and heuristics supported by each tool; heuristics refers to software vulnerabilities/errors, if they are supported by the tools and how these heuristics are defined. This is analysed in higher detail in section IV.

The last section of Table V aggregates information from KLEE and Owi documentation on SAT and SMT solvers. While SAT solvers handle *boolean satisfiability*, SMT expand on this concept into general formulas[44]. Examples of SMT solvers are STP and Z3 but there are also wrappers like MetaSMT and frontends as Smt.ml [45–48]. MiniSat is a SAT solver used by STP as the underlying SAT solver and Smt.ml also has MiniSat in its support roadmap [48, 49].

*F*eature	KLEE	Owi
Installation speed	5 (2)	2
Installation complexity	1 (4)	3
Ease-of-use (begginer)	4	2
Ease-of-use (experienced)	4	4
Heuristics support	Larger	Reduced
Heuristics definition	Hardcoded	Hardcoded
Supports E-ACSL	NO	YES
Supports MetaSMT	YES	NO
Supports STP	YES	NO
Supports Z3	YES	YES
Supports metaSMT solvers	YES	NO
Supports Smt.ml solvers	NO	YES
Supports MiniSat	YES	NO

TABLE V: Feature comparison

IV. EXPERIENCES AND RESULTS

In this chapter, multiple experiences using both KLEE and Owi tools will be described and analysed. We will start by describing the test environment in section IV-A, and then proceed to describe each experience in its own section. Finally,

a summary of all results and an overview of the experiences will be presented in section IV-K.

Each experience will showcase their capabilities and limitations. The samples 1 and 2 (sections IV-B and IV-C) were extracted from KLEE's online documentation and samples 3 through 5 (sections IV-D to IV-F) from Owi's GitHub repository. Samples 6 through 9 (sections IV-G to IV-J) were recovered from **(author?)** [36] blog. All samples were adapted with necessary changes to ensure compatibility with both tools.

(author?) [50] describe themselves as "a world-class player in software security" and as such they have many interactions in software security applications, such as symbolic execution. One of these interactions is a blog on KLEE where they go through the installation and usage of the tool to find bugs. On this blog, **(author?)** [36] presents 4 videos to demonstrate KLEE set of features and in video 3 they discuss vulnerabilities of the types:

- Global Buffer Overflow (Sample 6 - Global Buffer Overflow),
- Stack-based Buffer Overflow (Sample 7 - Stack-based Buffer Overflow),
- NULL pointer dereference (Sample 8 - NULL pointer dereference),
- Use after `free` (Sample 9 - Use after free).

Sections IV-G to IV-J will cover these vulnerabilities using both KLEE and Owi by instrumenting each source in the intended way.

A. Test Environment

To increase the results' credibility, all tests were run on the same machine, therefore sharing the same resources. The specifications of such machine consisted in a 12th Gen Intel Core processor (i7-1255U) of 10 cores (and a total of 12 threads) with a max clock speed of 4.70 GHz, and 20 GB (4 + 16) of DDR4 3200 MHz RAM.

This is the parent system setup with a Windows 11 installation; however, the test environment was created in a WSL environment within this machine which shares 4 cores and 8 GB of RAM available.

For simplicity of testing, all samples were organized by folders and subfolders: the parent folder aggregates by sample (1, 2, 3, etc.) and child folders aggregates files by tool, KLEE or Owi; a folder description is visible in **??** 21**??** 22.

Each child folder (e.g., sample1/klee, sample1/owi) contains its own 'main.c' file with the sample source code and will also contain any generated output by the tool. On KLEE runs, the `klee-out-n` (where `n` is the number of the run) and `klee-last` folders are created, and on Owi, the `owi-out`. There are also the compiled objects `main.bc` and `main.wasm` files for targets LLVM and WASM platforms, respectively.

```
.
├── sample1
│   ├── klee
│   └── owi
├── sample2
│   ├── klee
│   └── owi
├── sample3
│   ├── klee
│   └── owi
├── sample4
│   ├── klee
│   └── owi
├── sample5
│   ├── klee
│   └── owi
...
```

Listing 21: Samples folder structure

```
.
|
...
├── sample6
│   ├── klee
│   └──owi
├── sample7
│   ├── klee
│   └── owi
├── sample8
│   ├── klee
│   └── owi
└── sample9
    ├── klee
    └── owi
```

Listing 22: Samples folder structure

```
./sample1
├── klee
│   ├── klee-last -> klee-out-0
│   ├── klee-out-0
│   ├── klee-output.txt
│   ├── klee-time.txt
│   ├── main.bc
│   └── main.c
└── owi
    ├── main.c
    ├── main.wasm
    ├── owi-out
    ├── owi-output.txt
    └── owi-time.txt
```

Listing 23: Sample file structure

B. Sample 1 - Dry-run

The first sample (see **??**) was extracted from the KLEE's online documentation tutorial one [37]. **??** 24 shows how `klee_make_symbolic` is used to create a symbol from variable `a`. Its arguments are explained in subsection III-B3.

To achieve a similar result with Owi, the sources must be adapted as shown in **??** 25. On section IV-B the variable definition is replaced by a declaration with assignment to the output of function `owi_i32`; this routine is explained in greater detail in subsection section III-C3.

Running this sample did not return any detected problem for either tool, but that was expected as this sample only serves

as an introduction to the tool. Despite no results being found, this experience acts as a false positive test and both KLEE and Owi passed.

```
int main() {
  int a;
  klee_make_symbolic(&a, sizeof(a), "a");
  return get_sign(a);
}
```

Listing 24: Sample 1 instrumentation for KLEE

```
#include <owi.h>

// ...

int main() {
  int a = owi_i32();
  return get_sign(a);
}
```

Listing 25: Sample 1 instrumented for Owi

C. Sample 2 - Regular Expression Matcher

The second sample is also extracted from one of KLEE's tutorials, specifically the second one, where **(author?)** [38] introduces a handy argument to reduce unwanted noise and also describes error reports.

If we run the source with instrumentation for KLEE (**??**) as advised in the Running KLEE subsection (**??** 10), 6'677 test cases are created; this order of results isn't bearable for a developer to analyse and check if it is a true or a false positive. This can be reduced to 16 generated tests, without losing code coverage, by adding the argument '`--only-output-states-covering-new`' to the tool scan, which enables `klee` to skip states that do not cover new code.

When it comes to Owi, after replacing the tutorial 2 source with Owi directives for symbolic values as in **??** 28, a difference between the test sources is noticeable. While KLEE is able to treat arrays as a single value, Owi requires the definition of each position one by one, making them all individual symbols. The library contains a definition for a function named `owi_malloc` that resembles the native `malloc`, but its arguments are not explicit and the declarations in '`owi.h`' library does not contain any explicit type of documentation or even a comment.

However, if we have a deeper look at the available library, see **??**, it is possible to find a pattern. On **??**–**??** (of **??**), the method declarations are paired with a line that seems to add more information to the methods.

?? 26 shows the example of `owi_i8` function, where before its declaration there are two instructions: '`import_module("symbolic")`' and '`import_name("i8_symbol")`', both contain name references to symbols (and therefore symbolic value types). Furthermore, on **??**–**??** and **??**–**??** (**??**) these same instructions are present but with different content (see '`owi_malloc`' example on **??** 27) which makes it compelling to believe that this function has a different responsibility than that of informing `owi` of symbolic values.

```
__attribute__((import_module("symbolic"),
    ↪ import_name("i8_symbol"))) char
owi_i8(void);
```

Listing 26: Owi routine declaration - '`owi_i8`'

```
__attribute__((import_module("summaries"),
    ↪ import_name("alloc"))) void *
owi_malloc(void *, unsigned int);
```

Listing 27: Owi routine declaration - '`owi_malloc`'

```
// ...

#include <owi.h>

// ...

int main()
{
  // The input regular expression.
  char re[SIZE];

  // Make the input symbolic.
  for (int i = 0; i < 10; i++)
  {
    re[i] = owi_char();
  }

  // Try to match against a constant string "hello".
  match(re, "hello");

  return 0;
}
```

Listing 28: Sample 2 instrumented for Owi

D. Sample 3 - Maze

Owi's GitHub repository [32] also contains a directory with small examples to instruct users on how the tool subcommands work. For `sym`subcommand, the only available example is not only written in WebAssembly text format, but it also is a very simple example not worth converting to *C*. That being said, the `c` folder contains multiple examples of varying complexity. Section IV-D attempts to solve a maze, while sections IV-E and IV-F will work on two different approaches showcasing one feature of `owi c`.

This sample demonstrates an algorithm that attempts to solve a simple maze. The example is very raw and direct, but the code could be adapted to a function that accepts an array of characters, each character representing a movement instruction, and returns a success indicator using an integer value, zero for success and any non-zero value for failure. A movement instruction is defined using lower-case characters for all directions: the character to move upwards is '*w*', '*s*' for downwards, and '*a*' and '*d*' for left and right movements.

Running `owi` on the original sample (see **??**), returns one maze solution. Looking at the test case in **??** 29, the generated information is outputted in a counter-intuitive sorting: sections IV-D to IV-D are formatted into 4 columns, a first describing stating the line refers to a symbol, a second with the symbol's name, another describing the symbol type, and lastly the symbol's value.

```
Assert failure: false
model {
  symbol symbol_0 i8 115
  symbol symbol_1 i8 100
  symbol symbol_10 i8 100
  symbol symbol_11 i8 119
  symbol symbol_12 i8 111
  symbol symbol_13 i8 0
  symbol symbol_14 i8 0
  symbol symbol_15 i8 0
  symbol symbol_16 i8 0
  symbol symbol_17 i8 0
  symbol symbol_18 i8 0
  symbol symbol_19 i8 0
  symbol symbol_2 i8 100
  symbol symbol_20 i8 0
  symbol symbol_21 i8 0
  symbol symbol_22 i8 0
  symbol symbol_23 i8 0
  symbol symbol_24 i8 0
  symbol symbol_25 i8 0
  symbol symbol_26 i8 0
  symbol symbol_27 i8 0
  symbol symbol_3 i8 119
  symbol symbol_4 i8 100
  symbol symbol_5 i8 100
  symbol symbol_6 i8 100
  symbol symbol_7 i8 100
  symbol symbol_8 i8 115
  symbol symbol_9 i8 100
}
Reached problem!
```

Listing 29: Sample 3 - Owi output

If we have a look at the second column of sections IV-D to IV-D (**??** 29), we notice an odd ordering of the columns: the expected output would be a sorting where, for names following the template 'symbol˙*n*' where *n* is an index starting at 0, the output is sorted by *n* in increasing order, starting from 0. However, the actual output seems to be sorted using some type of string sorting where strings of different sizes are compared character by character up to the last character of the smaller string, e.g., *symbol˙1* is the same length as *symbol˙2*, but both are smaller than *symbol˙10*: *symbol˙10* is equal to *symbol˙1* but has a higher length and therefore comes after; *symbol˙2* is greater than *symbol˙1* **and** *symbol˙10*, therefore it goes **after** *symbol˙10*.

On **??** of **??** we can see that each position of the char array was defined using the function `owi_char` which informs Owi of a new symbolic variable of the type `char`. Despite this, the symbol's type seems to be interpreted to `i8` where the number '8' represents 8 bits of memory, or 1 byte, the same size as the type `char`.

Considering that the symbolic type of all symbols is defined as an integer type, their values will also be of integer type. Knowing this, and knowing that the instructions expected for the maze algorithm are one of $\{w, a, s, d\}$, the symbol integer values must be converted to their character equivalents using an ASCII table which are $\{119, 97, 115, 100\}$, respectively. There is an additional ASCII code (111 - '*o*') that does not seem to be included in any of the expected inputs; this might be due to a faulty algorithm implementation, which is not the expected outcome of using Owi, but still an acceptable outcome. Another possible explanation is that the algorithm is correct and this unexpected input is an Owi bug.

To test this algorithm using KLEE, the sources must be instrumented with the dedicated functions. Section IV-D were updated to support KLEE, and one significant difference between Owi is section IV-D where the iteration of one array, and initialization of all elements, was replaced by one instruction that takes the array address and its size, doing all computations and memory accesses correctly.

```c
#include <klee/klee.h>

// ...

int main (void) {

  int x = 1;
  int y = 1;
  maze[y][x]='X';

  char program[ITERS];
  klee_make_symbolic(&program, sizeof program, "
    ↪ program");

  int old_x = x;
  int old_y = y;

  for (int i = 0; i < ITERS; i++) {
    // ...

    if (maze[y][x] == '#') {
      // TODO: print the result
      klee_assert(0);
      return 0;
    }

    // ...
  }
  return 1;
}
```

Listing 30: Sample 3 instrumented for KLEE

Looking at the last lines of *stdout* information generated by KLEE, **??** 31, some relevant data can be extracted: firstly, one error was detected and KLEE knows exactly where the bug occurred ('`main.c:55`') and which type of failure it is ('`ASSERTION FAIL: 0`'); this error will no longer be reported by any other path that may trigger this same error at this same location. Additionally, we're told 306 test cases were generated but only 305 completed paths were traced - the known bug that fails execution on section IV-D of **??** 30.

```
KLEE: ERROR: main.c:55: ASSERTION FAIL: 0
KLEE: NOTE: now ignoring this error at this location

KLEE: done: total instructions = 47849
KLEE: done: completed paths = 305
KLEE: done: partially completed paths = 4
KLEE: done: generated tests = 306
```

Listing 31: Sample 3 - KLEE stdout

If we examine the test case created for the detected BUG (**??** 32), we notice on section IV-D that it matches the expected outcome of using only one symbol definition instruction *versus* defining all array positions. On sections IV-D to IV-D the

information for "*object 0*" is displayed: this object is the first object declared in the program, but if this is not enough information to know what it refers to, we can always rely on section IV-D where the object's name is shown between single quotes. On section IV-D we're told the object size, and we know it matches the 28 positions of the array (**????** on **??**).

```
$ ktest-tool klee-last/test000113.ktest

ktest file : 'klee-last/test000113.ktest'
args       : ['main.bc']
num objects: 1
object 0: name: 'program'
object 0: size: 28
object 0: data: b'sddwddddsddw\xff\xff\xff\xff\xff\
    ↪ xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff'
object 0: hex : 0
    ↪ x736464776464646473646477ffffffffffffffffffffffffffffffff
    ↪
object 0: text: sddwddddsddw.............
```

Listing 32: Sample 3 - Display KLEE generated test case for detected bug using ktest-tool

KLEE shows the variable values in five possible data types: a concrete object data as Python byte string, hexadecimal values in a string, signed and unsigned integer values, and text value. All of this information was extracted from the `ktest-tool` "help page", visible on **??** 33 , on which is also available that all non-printable characters are replaced by dots ('.') on text displays of the symbolic values.

```
$ ktest-tool --help
usage: ktest-tool [-h] [--trim-zeros] [--extract
    ↪ name] file [file ...]

positional arguments:
  file            a .ktest file

options:
  -h, --help      show this help message and exit
  --trim-zeros    trim trailing zeros
  --extract name  write binary value of object into
    ↪ file

output description:
  A .ktest file comprises a file header and a list
    ↪ of memory objects.
  Each object holds concrete test data for a
    ↪ symbolic memory object.
  As no type information is stored, ktest-tool
    ↪ outputs data in
  different representations.

  ktest file header:
    ktest file: path to ktest file
    args: program arguments
    num objects: number of memory objects
  memory object:
    name: object name
    size: object size
    data: concrete object data as Python byte string
    hex: data as hex string
    int: data as integer if size is 1, 2, 4, 8 bytes
    uint: data as unsigned integer if size is 1, 2,
    ↪ 4, 8 bytes
    text: data as ascii text, '.' for non-printable
    ↪ characters

example:
  > ktest-tool klee-last/test000003.ktest
  ktest file : 'klee-last/test000003.ktest'
  args       : ['get_sign.bc']
  num objects: 1
  object 0: name: 'a'
  object 0: size: 4
  object 0: data: b'\x00\x00\x00\x80'
  object 0: hex : 0x00000080
  object 0: int : -2147483648
  object 0: uint: 2147483648
  object 0: text: ....
```

Listing 33: ktest-tool "help page"

E. Sample 4 - Prime numbers

In this section, we analyse a sample based on **??** where we remove **??**–**??** resulting in **??** 34. This is done to simplify the source code and focus on the symbolic execution capabilities of both tools.

Owi's instrumentation is focused only in defining the variable '*n*' as a symbolic variable and constraining the expected values to be in the interval `[2, MAX_SIZE]` where `MAX_SIZE` is known to be constant and set to 100 (section IV-E in **??** 34). KLEE does the same instrumentation, but naturally using its own functions, in **??** 35 (section IV-E).

Both Owi and KLEE return no results, which in this case contributes to the false positives' evaluation - both tools passed this test.

```c
#define MAX_SIZE 100

#include <owi.h>
#include <stdlib.h>

void primes(int *is_prime, int n) {
    for (int i = 0; i < n; ++i)
        is_prime[i] = 1;
    for (int i = 2; i * i < n; ++i) {
        if (!is_prime[i]) continue;
        for (int j = i; i * j < n; ++j) {
            is_prime[i * j] = 0;
        }
    }
}

int main(void) {
    int *is_prime;
    is_prime = malloc(MAX_SIZE * sizeof(int));

    int n = owi_i32();
    owi_assume(n >= 2);
    owi_assume(n <= MAX_SIZE);

    primes(is_prime, n);
    free(is_prime);
    return 0;
}
```

Listing 34: Sample 4 instrumented for Owi

```c
#define MAX_SIZE 100

#include <klee/klee.h>
#include <stdlib.h>

void primes(int *is_prime, int n) {
    for (int i = 0; i < n; ++i)
```

```
        is_prime[i] = 1;
    for (int i = 2; i * i < n; ++i) {
        if (!is_prime[i])
            continue;
        for (int j = i; i * j < n; ++j) {
            is_prime[i * j] = 0;
        }
    }
}

int main(void)
{
    int *is_prime;
    is_prime = malloc(MAX_SIZE * sizeof(int));

    int n;
    klee_make_symbolic(&n, sizeof n, "variable n");
    klee_assume(n >= 2);
    klee_assume(n <= MAX_SIZE);

    primes(is_prime, n);
    free(is_prime);
    return 0;
}
```

Listing 35: Sample 4 instrumented for KLEE

F. Sample 5 - Prime numbers (Frama-C's E-ACSL)

This section will work with **??** without modifications. Owi makes use of this example to introduce its integration with the Frama-C's E-ACSL plugin which is stated to automatically translate 'an annotated C program into another program that detects the violated annotations at runtime' [51]. This feature can help developers prevent bug introduction and/or regressions of business logic bugs, it does not help in detecting vulnerable code.

?? is annotated with E-ACSL instructions that will define conditions that the plugin will enforce both on input and output values. These annotations are contained in a block comment like so: `/*@ ... */`. The keyword '`requires`' is used define input value conditions, and '`ensures`' is for output values.

??–**??** in **??** and **??** 37 (for Owi and KLEE, respectively), define how the function `primes` will only receive values for '`n`' that are in the interval `[2, MAX_SIZE]` and values for '`is_prime`' such that all memory locations are safe to read and write: assuming '`x`' is the memory address pointed by `is_prime`, the interval of addresses `[x, x + (n - 1)]` is safe to operate.

Running owi on this sample returns the result on **??** 38 defines clearly how the algorithm fails for `n == 2`. By default, owi stops execution on the first detected problem, and in order to find multiple issues it must be passed the option '`--deterministic-result-order`' which also guarantees that the results are found in a consistent order. With this new argument owi detects 99 problems, one for each value in the interval $[2, 100]$ (`MAX_SIZE = 100`).

The error detected by KLEE was the result of an insufficient compilation of the sources, and the outputted error was '`failed external call: __e_acsl_memory_init`'. Even though this is not the expected outcome of the scan, it is a valid result since the program would fail in a concrete execution; however, we will consider this a False Positive since we did not want this result and the run did not execute as expected.

```
KLEE: ERROR: main.instrumented.c:695: failed
    ↪ external call: __e_acsl_memory_init
KLEE: NOTE: now ignoring this error at this location

KLEE: done: total instructions = 12021
KLEE: done: completed paths = 0
KLEE: done: partially completed paths = 1
KLEE: done: generated tests = 1
```

Listing 36: Sample 5 - KLEE output

```
#define MAX_SIZE 100

#include <klee/klee.h>
#include <stdlib.h>

/*@ requires 2 <= n <= MAX_SIZE;
    requires \valid(is_prime + (0 .. (n - 1)));
    ensures  \forall integer i; 0 <= i < n ==>
        (is_prime[i] <==>
            (i >= 2 && \forall integer j; 2 <= j < i
    ↪  ==> i % j != 0));
*/
void primes(int *is_prime, int n) {
    for (int i = 0; i < n; ++i)
        is_prime[i] = 1;
    for (int i = 2; i * i < n; ++i) {
        if (!is_prime[i]) continue;
        for (int j = i; i * j < n; ++j) {
            is_prime[i * j] = 0;
        }
    }
}

int main(void) {
    int *is_prime;
    is_prime = malloc(MAX_SIZE * sizeof(int));

    int n;
    klee_make_symbolic(&n, sizeof n, "variable n");
    klee_assume(n >= 2);
    klee_assume(n <= MAX_SIZE);

    primes(is_prime, n);
    free(is_prime);
    return 0;
}
```

Listing 37: Sample 5 instrumented for KLEE

```
Assert failure: false
model {
  symbol symbol_0 i32 2
}
Reached 1 problem!
```

Listing 38: Sample 5 - Owi output

G. Sample 6 - Global Buffer Overflow

On this section, we will be covering an example of a Global Buffer Overflow vulnerability and how these problems can be detected using KLEE and Owi.

On **??**, we can see how the `test1` input variable `arr` is safely marked as symbolic with **????**, using the KLEE instrumentation methods. **??** makes the array safe for string operations like '`strcmp`' on **??**.

The second instrumentalization is done on **??** 39. Because the symbolic input is an array, each position must be initialized (see sections IV-G to IV-G) and because this is an array of character values (a string) we're telling Owi the string value is suffixed by '\0' using `owi_assume` function and passing the condition `arr[9] == '\0'` as its argument (section IV-G). The string is 'closed' so that it is safe for string operations, such as '`strcmp`' on section IV-G, and it isn't reported as a problem.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <owi.h>

int global_arr[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
int test1(char *buf)
{
  int i = 5;
  if (strcmp(buf, "BUG!") == 0)
  {
    i += 20;
  }
  return global_arr[i];
}

int main(int argc, char const *argv[])
{
  char arr [10];
  for (int i = 0; i < 10; i++) {
    arr[i] = owi_char();
  }
  owi_assume(arr[9] == '\0');

  test1(arr);

  return 0;
}
```

Listing 39: Sample 6 instrumented for Owi

Starting with klee execution, the results are found very quickly and only 6 paths are traced, one of which is partial and an error detection. **??** 40 also shows on section IV-G the type of error ("`memory error: out of bound pointer`") and on which file and line the error occurred ("`main.c:14`"). All unique errors generate a test case of its own and the one detected for this sample is expressed in detail on **??** 41.

On the other hand, owi displays "`All OK`" after scanning the sources stating it did not find the vulnerability.

```
KLEE: ERROR: main.c:14: memory error: out of bound
    ↪ pointer
KLEE: NOTE: now ignoring this error at this location

KLEE: done: total instructions = 12493
KLEE: done: completed paths = 5
KLEE: done: partially completed paths = 1
KLEE: done: generated tests = 6
```

Listing 40: Sample 6 - KLEE output

```
ktest file : './sample6/klee/klee-last/test000003.
    ↪ ktest'
args       : ['main.bc']
num objects: 1
object 0: name: 'arr'
object 0: size: 10
object 0: data: b'BUG!\x00\xff\xff\xff\xff\x00'
object 0: hex : 0x4255472100ffffffff00
object 0: text: BUG!......
```

Listing 41: Sample 6 - KLEE test case for the detected error

H. Sample 7 - Stack-based Buffer Overflow

Differently from Sample 6 - Global Buffer Overflow, Stack-based Buffer Overflow vulnerabilities look at scoped and dynamically allocated variables like the ones on line 8 in both KLEE and Owi samples (**??** and **??** 44); the samples' instrumentation is identical to what is seen on Sample 6 - Global Buffer Overflow.

Running KLEE results in a very similar result to that seen on section IV-G, once again 6 generated tests, one of which triggered an execution error shown in detail on **??** 43. This time, the error occurs on **??** of file main (**??**). Owi fails to find the vulnerability, again.

```
KLEE: NOTE: KLEE: ERROR: main.c:26: memory error:
    ↪ out of bound pointer
now ignoring this error at this location

KLEE: done: total instructions = 12566
KLEE: done: completed paths = 5
KLEE: done: partially completed paths = 1
KLEE: done: generated tests = 6
```

Listing 42: Sample 7 - KLEE output

```
ktest file : './sample7/klee/klee-last/test000003.
    ↪ ktest'
args       : ['main.bc']
num objects: 1
object 0: name: 'arr'
object 0: size: 10
object 0: data: b'BUG!\x00\xff\xff\xff\xff\x00'
object 0: hex : 0x4255472100ffffffff00
object 0: text: BUG!......
```

Listing 43: Sample 7 - KLEE test case for the detected error

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <owi.h>

int test2(char *buf)
{
  int *dyn_mem_arr = malloc(sizeof(int) * 10);
  dyn_mem_arr[0] = 0;
  dyn_mem_arr[1] = 1;
  dyn_mem_arr[2] = 2;
  dyn_mem_arr[3] = 3;
  dyn_mem_arr[4] = 4;
  dyn_mem_arr[5] = 5;
  dyn_mem_arr[6] = 6;
  dyn_mem_arr[7] = 7;
  dyn_mem_arr[8] = 8;
  dyn_mem_arr[9] = 9;

  int i = 5;
  if (strcmp(buf, "BUG!") == 0)
  {
    i += 20;
  }

  int return_value = dyn_mem_arr[i];
  free(dyn_mem_arr);
```

```
28    return return_value;
29  }
30
31  int main(int argc, char const *argv[])
32  {
33    char arr[10];
34    for (int i = 0; i < 10; i++)
35    {
36      arr[i] = owi_char();
37    }
38    owi_assume(arr[9] == '\0');
39
40    test2(arr);
41    return 0;
42  }
```

Listing 44: Sample 7 - Owi instrumentation

I. Sample 8 - NULL pointer dereference

To test NULL pointer dereference bugs, a small string array (‘`char *global_string[]`’ on section IV-I) is initialized and used in function ‘`test3`’ where on section IV-I will trigger an error when the input is favorable.

After the samples are instrumented, the tools are run and a pattern begins to show where KLEE detects the vulnerability but Owi does not. See **??** and **??** 45 for KLEE and Owi instrumented samples, respectively, **??** 46 for KLEE terminal output and **??** 47 for the generated test case.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <owi.h>
5
6  char *global_string[] = {
7    "string1",
8    "string2",
9    "string3",
10   NULL,
11 };
12
13 int test3(char *buf)
14 {
15   int i = 1;
16   if (strcmp(buf, "BUG!") == 0)
17   {
18     i = 3;
19   }
20
21   char c = *(global_string[i]);
22   if (c == 's')
23     return 0;
24   return 1;
25 }
26
27 int main(int argc, char const *argv[])
28 {
29   char arr[10];
30   for (int i = 0; i < 10; i++) {
31     arr[i] = owi_char();
32   }
33   owi_assume(arr[9] == '\0');
34
35   test3(arr);
36   return 0;
37 }
```

Listing 45: Sample 8 - Owi instrumentation

```
1 KLEE: ERROR: main.c:21: memory error: out of bound
      ↪ pointer
2 KLEE: NOTE: now ignoring this error at this location
3
4 KLEE: done: total instructions = 12549
5 KLEE: done: completed paths = 5
6 KLEE: done: partially completed paths = 1
7 KLEE: done: generated tests = 6
```

Listing 46: Sample 8 - KLEE output

```
1 ktest file : './sample8/klee/klee-last/test000003.
      ↪ ktest'
2 args       : ['main.bc']
3 num objects: 1
4 object 0: name: 'arr'
5 object 0: size: 10
6 object 0: data: b'BUG!\x00\xff\xff\xff\xff\x00'
7 object 0: hex : 0x4255472100ffffffff00
8 object 0: text: BUG!......
```

Listing 47: Sample 8 - KLEE test case for the detected error

J. Sample 9 - Use after free

Finally, ‘Use after free’ runtime errors can also become very hard to detect by the human eye when code complexity increases; ‘`test4`’ function presents a simple case of this vulnerability to showcase KLEE and Owi capabilities. **??** and **??** 51 illustrate instrumented sources for each symbolic execution engine.

KLEE detected the bug on **??**, generated 6 tests, and distinguished the error from the ones on sections IV-G to IV-I. KLEE’s output (**??** 48) shows that the memory error is of type ‘`double free`’ on section IV-J, and the generated test case (**??** 49) shows the expected input value.

```
1 KLEE: ERROR: main.c:14: memory error: double free
2 KLEE: NOTE: now ignoring this error at this location
3
4 KLEE: done: total instructions = 12484
5 KLEE: done: completed paths = 5
6 KLEE: done: partially completed paths = 1
7 KLEE: done: generated tests = 6
```

Listing 48: Sample 9 - KLEE output

```
1 ktest file : './sample9/klee/klee-last/test000003.
      ↪ ktest'
2 args       : ['main.bc']
3 num objects: 1
4 object 0: name: 'arr'
5 object 0: size: 10
6 object 0: data: b'BUG!\x00\xff\xff\xff\xff\x00'
7 object 0: hex : 0x4255472100ffffffff00
8 object 0: text: BUG!......
```

Listing 49: Sample 9 - KLEE test case for the detected error

While KLEE detected the double free as any other code issue, Owi reported the problem as an internal error and its respective stack trace as displayed in **??** 50. This behaviour suggests the tool does not properly support this issue and might cause the scan to miss other issues because of this internal error.

```
owi: internal error, uncaught exception:
     Failure("Memory leak double free")
     Raised at Stdlib__Domain.join in file "domain.
    ↪ ml", line 296, characters 16-24
     Called from Stdlib__Array.iter in file "array.
    ↪ ml", line 113, characters 31-48
     Called from Owi__Wq.read_as_seq.(fun) in file "
    ↪ src/data_structures/wq.ml", line 17,
    ↪ characters 4-16
     Called from Owi__Cmd_sym.cmd.
    ↪ print_and_count_failures in file "src/cmd/
    ↪ cmd_sym.ml", line 99, characters 10-20
     Called from Owi__Cmd_sym.cmd in file "src/cmd/
    ↪ cmd_sym.ml", line 130, characters 15-49
     Called from Cmdliner_term.app.(fun) in file "
    ↪ cmdliner_term.ml", line 24, characters 19-24
     Called from Cmdliner_eval.run_parser in file "
    ↪ cmdliner_eval.ml", line 35, characters 37-44
```

Listing 50: Sample 9 - Owi output

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <owi.h>

int test4(char *buf)
{
  char *var = malloc(10);
  int i = 5;
  if (strcmp(buf, "BUG!") == 0)
  {
    free(var);
  }
  free(var);

  return 1;
}

int main(int argc, char const *argv[])
{
  char arr[10];
  for (int i = 0; i < 10; i++) {
    arr[i] = owi_char();
  }
  owi_assume(arr[9] == '\0');

  test4(arr);
  return 0;
}
```

Listing 51: Sample 9 - Owi instrumentation

K. Results

Both KLEE and Owi require compiled sources, however Owi can abstract that process for the user if they are testing C source code. To avoid our statistics to include resources spent in source compilation, we will compile our sources for Owi using the same command they do in their abstraction. Please note that the comparisons in Table V - Feature comparison take only symbolic execution features into consideration.

This section discusses **Accuracy** and **Performance** metrics such as:

Accuracy Metrics

- **TP - True Positives**
  Number of **expected** vulnerabilities **reported**.
- **TN - True Negatives**
  Number of **unexpected** vulnerabilities **not reported**.
- **FP - False Positives**
  Number of **unexpected** vulnerabilities **reported**.
- **FN - False Negatives**
  Number of **expected** vulnerabilities **not reported**.
- **Youden Index or Youden's J statistic**
  The accuracy metric used in OWASP Benchmark [52] is the following
  $J = TPR - FPR$

$$= (\frac{TP}{TP+FN})^a - (\frac{FP}{TN+FP})^b$$

$$= \frac{TP*TN - FP*FN}{(TP+FN)(TN+FP)}$$

[a] TPR - True Positive Rate
[b] FPR - False Positive Rate

Performance Metrics

- **Paths traced**
  Branches of execution created to navigate.
- **CPU (%)**
  The CPU used by the process, in percentage.
- **Memory (KB)**
  The 'maximum resident set size' (very similar to peak memory) used by the process, in kilobytes.
- **Time (s)**
  The execution time of the process, in seconds.

Accuracy metrics were extracted by manually inspecting each scan output, and performance metrics such as CPU, memory and time of execution were measured using the GNU `time` program (see **??** 52). "**Paths traced**" was also extracted manually similarly to accuracy metrics. There is also the "**Success**" metric that indicates whether the sample was successfully analysed by the test tool, without regard for results.

```
# to avoid confusion with bash keyword 'time'
# use the full path given to you by `which time`

time -v klee main.bc

time -v owi sym main.wasm
```

Listing 52: Extraction of performance metrics

After executing both engines on all test samples and extracting the results from each of them, Tables VI and VII we're created. With these metrics properly formatted, calculating the "**Youden Index**" for both tools is easier, higher is better.

It is apparent how KLEE detected much more vulnerabilities than Owi, which plays in its favour, and the calculations on equations (2) and (3) reflect this statement, KLEE scores approximately 54% while Owi stands on 29% of bug detection accuracy.

If we look at the Youden Index interpretation guide from OWASP Benchmark [52] shown in Figure 2, we can see that KLEE scores highly (87.5%) in the y axis (TPR), while Owi

scores poorly (28.5%). Both tools score low on the x axis (FPR), with KLEE at 33.3% and Owi at 0%. This means that KLEE is very good at detecting vulnerabilities, but also has a higher rate of false positives, while Owi is not very good at detecting vulnerabilities, but has a low rate of false positives. A larger set of samples would be needed to draw more concrete conclusions.

Sample	Success	TP	TN	FP	FN
1	YES	0	1	0	0
2	YES	2	0	0	0
3	YES	1	0	0	0
4	YES	0	1	0	0
5	YES[a]	0	0	1	1
6	YES	1	0	0	0
7	YES	1	0	0	0
8	YES	1	0	0	0
9	YES	1	0	0	0

TABLE VI: KLEE accuracy metrics

[a]KLEE was able to run, but it wasn't able to make use of Frama-C's E-ACSL capabilities

Sample	Success	TP	TN	FP	FN
1	YES	0	1	0	0
2	YES	0	0	0	1
3	YES	1	0	0	0
4	YES	0	1	0	0
5	YES	1	0	0	0
6	YES	0	0	0	1
7	YES	0	0	0	1
8	YES	0	0	0	1
9	YES	0	0	0	1

TABLE VII: Owi accuracy metrics

$$J = (\frac{7}{7+1})^a - (\frac{1}{2+1})^b \approx (0.875) - (0.333) \approx 0.54 \quad (2)$$

KLEE Youden Index

[a]TPR - True Positive Rate
[b]FPR - False Positive Rate

$$J = (\frac{2}{2+5})^a - (\frac{0}{2+0})^b \approx (0.285) - (0) \approx 0.29 \quad (3)$$

Owi Youden Index

[a]TPR - True Positive Rate
[b]FPR - False Positive Rate

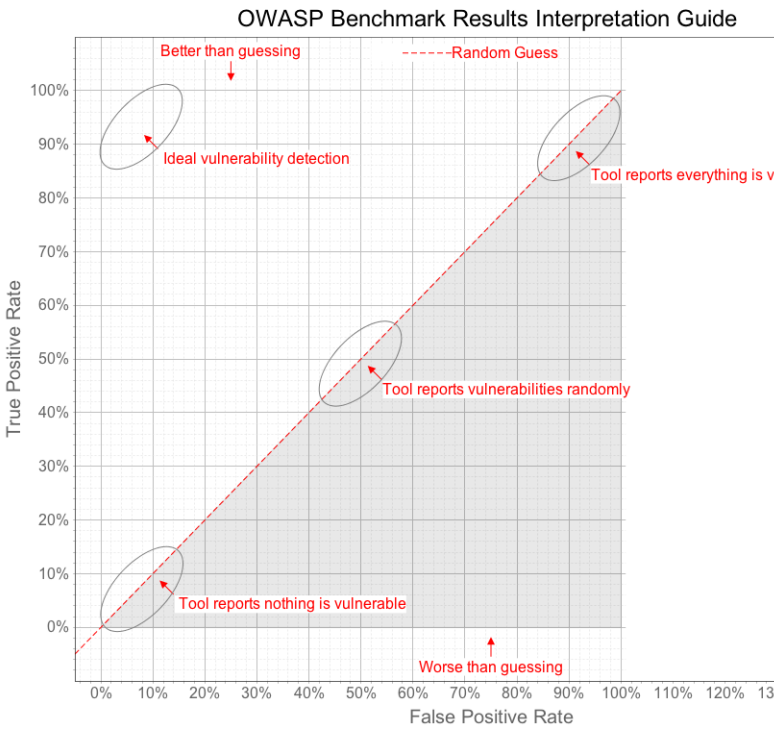

Fig. 2: OWASP Benchmark Scoring Guide - Youden Index interpretation*

*

By running KLEE and Owi on the same set of samples, some differences are visible in their outputs and in their performances (see Tables VIII and IX). KLEE was overall slower, with the only exception being the primes example, but was also the example that consumed less CPU but also Memory on all samples.

Unfortunately, Owi does not seem to output information regarding the travelled states, even after looking at debug logs (which depict execution instructions, but nothing was found to support a number of traversed paths).

An example of how the performance between these tools differ is with sample 3, a test where KLEE traces 306 paths. KLEE reported results consuming nearly half the CPU resources and approximately 73% of the memory usage and 35% of the time spent, that Owi required to run and return no results on the sample.

Sample 2 shows an even greater difference; KLEE spends nearly half the CPU and 19% of the memory usage when compared to Owi, and does that in 1% of the time it took Owi to analyse the same source. This scenario might be a clear indicator of Owi limitations.

Sample	Success	Paths traced	CPU (%)	Memory (KB)	Time (s)[2]
1	YES	3	99	98'156	0.29
2	YES	6'675	101	139'680	12.93
3	YES	305	93	99'152	0.72
4	YES	99	98	99'504	2.93
5	YES	1	81	98'036	0.39
6	YES	5	102	98'920	0.30
7	YES	5	87	97'536	0.37
8	YES	5	96	98'248	0.33
9	YES	5	96	99'236	0.32

TABLE VIII: KLEE performance metrics

Sample	Success	Paths traced	CPU (%)	Memory (KB)	Time (s)[10]
1	YES	N/A	108	110'000	0.05
2	YES	N/A	200	725'892	901.51
3	YES	N/A	199	134'928	2.10
4	YES	N/A	204	145'676	9.90
5	YES	N/A	205	151'800	11.13
6	YES	N/A	116	120'172	0.06
7	YES	N/A	115	119'172	0.06
8	YES	N/A	118	118'220	0.05
9	YES	N/A	123	121'040	0.05

TABLE IX: Owi performance metrics

REFERENCES

REFERENCES

[1] R. Croft, D. Newlands, Z. Chen, and A. M. Babar, "An empirical study of rule-based and learning-based approaches for static application security testing," *International Symposium on Empirical Software Engineering and Measurement*, 10 2021. [Online]. Available: http://eprints.whiterose.ac.uk/95628/

[2] M. Sharma, "Review of the benefits of dast (dynamic application security testing) versus sast," *International Journal of Management and Engineering Research*, vol. 1, pp. 1–4, 6 2021. [Online]. Available: http://www.ijmer.org/index.php/journal/article/view/2

[3] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research." *MIS Quarterly*, vol. 28, pp. 75–75–105, 2004.

[4] L. Andrès, F. Marques, A. Carcano, P. Chambart, J. F. F. dos Santos, and J.-C. Filliâtre, "Owi: Performant parallel symbolic execution made easy, an application to webassembly," *The Art, Science, and Engineering of Programming*, vol. 9, 10 2024. [Online]. Available: https://hal.science/hal-04627413

[5] A. Hoffman, *Web Application Security*. O'Reilly Media, Inc., 2024.

[6] F. M. Tudela, J.-R. B. Higuera, J. B. Higuera, J.-A. S. Montalvo, and M. I. Argyros, "On combining static, dynamic and interactive analysis security testing tools to improve owasp top ten security vulnerability detection in web applications," *mdpi.com*, 12 2020.

[7] L. Dencheva, "Comparative analysis of static application security testing (sast) and dynamic application security testing (dast) by using open-source web application penetration," Master's thesis, National College of Ireland, 8 2022. [Online]. Available: https://norma.ncirl.ie/id/eprint/5956

[8] OWASP Foundation. (2024) OWASP Benchmark — Scoring. [Online]. Available: https://owasp.org/www-project-benchmark/#div-scoring

[9] ——. (2023) OWASP DevSecOps Guideline - v-0.2 — 2-b - Dynamic Application Security Testing. [Online]. Available: https://owasp.org/www-project-devsecops-guideline/latest/02b-Dynamic-Application-Security-Testing

[10] S. Gupta and N. Gayathri, "Study of the software development life cycle and the function of testing," *International Interdisciplinary Humanitarian Conference for Sustainability, IIHC 2022 - Proceedings*, pp. 1270–1275, 2022.

[11] M. Alenezi and Y. Javed, "Open source web application security: A static analysis approach," *IEEE*, pp. 1–5, 2016.

[12] T. Rangnau, R. V. Buijtenen, F. Fransen, and F. Turkmen, "Continuous security testing: A case study on integrating dynamic security testing tools in ci/cd pipelines," *Proceedings - 2020 IEEE 24th International Enterprise Distributed Object Computing Conference, EDOC 2020*, pp. 145–154, 10 2020.

[13] OWASP Foundation. (2023) OWASP DevSecOps Guideline - v-0.2 — 2-a - Static Application Security Testing. [Online]. Available: https://owasp.org/www-project-devsecops-guideline/latest/02a-Static-Application-Security-Testing

[14] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, pp. 385–394, 7 1976.

[15] A. Vafaei, N. Hooten, M. Tehranipoor, and F. Farahmandi, "Symba: Symbolic execution at c-level for hardware trojan activation," *Proceedings - International Test Conference*, pp. 223–232, 2021.

[16] R. David, S. Bardin, T. D. Ta, J. Feist, L. Mounier, M. L. Potet, and J. Y. Marion, "Binsec/se: A dynamic symbolic execution toolkit for binary-level analysis," *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering, SANER 2016*, vol. 1, pp. 653–656, 5 2016.

[17] F. Gritti, L. Fontana, E. Gustafson, F. Pagani, A. Continella, C. Kruegel, and G. Vigna, "Symbion: Interleaving symbolic with concrete execution," *2020 IEEE Conference on Communications and Network Security, CNS 2020*, 6 2020.

[18] G. Zhang, Z. Chen, Z. Shuai, Y. Zhang, and J. Wang, "Synergizing symbolic execution and fuzzing by function-level selective symbolization," *Proceedings - Asia-Pacific Software Engineering Conference, APSEC*, vol. 2022-December, pp. 328–337, 12 2022.

[19] D. Kroening and O. Strichman, *Decision Procedures - An Algorithmic Point of View*, 1st ed. Springer Berlin, Heidelberg, 4 2008.

[20] C. Cadar, D. Dunbar, and D. Engler, "Klee: Unassisted

and automatic generation of high-coverage tests for complex systems programs," *KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs*, 2008. [Online]. Available: https://purl.stanford.edu/fx501rc2898

[21] P. Godefroid, "Fuzzing: Hack, art, and science," *Communications of the ACM*, vol. 63, pp. 70–76, 1 2020.

[22] E. J. Schwartz, T. Avgerinos, and D. Brumley, "All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask)," *Proceedings - IEEE Symposium on Security and Privacy*, pp. 317–331, 2010.

[23] K. Luckow. (2017, 7) ksluckow/awesome-symbolic-execution. A curated list of awesome symbolic execution resources including essential research papers, lectures, videos, and tools. [Online]. Available: https://github.com/ksluckow/awesome-symbolic-execution

[24] LLVM Project. The LLVM Compiler Infrastructure Project. [Online]. Available: https://llvm.org/

[25] Java Pathfinder. javapathfinder/jpf-symbc: Symbolic PathFinder. [Online]. Available: https://github.com/javapathfinder/jpf-symbc

[26] ——. javapathfinder/jpf-core Wiki. [Online]. Available: https://github.com/javapathfinder/jpf-core/wiki

[27] Samsung. Samsung/jalangi2: Dynamic analysis framework for JavaScript. [Online]. Available: https://github.com/Samsung/jalangi2

[28] G. Li, E. Andreasen, and I. Ghosh, "SymJS: automatic symbolic testing of JavaScript web applications," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. New York, NY, USA: ACM, Nov. 2014.

[29] Dependable Systems Lab. Cloud9 - Automated Software Testing at Scale. [Online]. Available: https://dslab.epfl.ch/research/cloud9/

[30] C. G. do Val, "Conflict-driven symbolic execution," Ph.D. dissertation, University of British Columbia, 2014.

[31] ——. Kite: A conflict-driven symbolic execution tool. [Online]. Available: https://www.cs.ubc.ca/labs/isd/Projects/Kite/

[32] Formal Security for Web Technologies. OCamlPro/owi: WebAssembly Swissknife & cross-language bugfinder. [Online]. Available: https://github.com/OCamlPro/owi

[33] Fermyon. WebAssembly Language Support Matrix — Fermyon Developer. [Online]. Available: https://developer.fermyon.com/wasm-languages/webassembly-language-support

[34] OWASP Foundation. WebGoat/WebGoat: WebGoat is a deliberately insecure application. [Online]. Available: https://github.com/WebGoat/WebGoat

[35] ——. OWASP WebGoat. [Online]. Available: https://owasp.org/www-project-webgoat/

[36] ADALogics. Symbolic execution with KLEE: From installation and introduction to bug-finding in open source software. [Online]. Available: https://adalogics.com/blog/symbolic-execution-with-klee

[37] The KLEE Team. Tutorial One · KLEE. [Online]. Available: https://klee-se.org/tutorials/testing-function

[38] ——. Tutorial Two · KLEE. [Online]. Available: https://klee-se.org/tutorials/testing-regex

[39] ——. Building KLEE with LLVM 13. [Online]. Available: https://klee-se.org/build/build-llvm13/

[40] N. M. d. Sabino, "Automatic vulnerability detection: Using compressed execution traces to guide symbolic execution," Master's thesis, Instituto Superior Técnico, 2019.

[41] WebAssembly. WebAssembly. [Online]. Available: https://webassembly.org

[42] ——. Use Cases - WebAssembly. [Online]. Available: https://webassembly.org/docs/use-cases/

[43] TIOBE. TIOBE Index - TIOBE. [Online]. Available: https://www.tiobe.com/tiobe-index/

[44] RBC Borealis. Tutorial #9: SAT Solvers I: Introduction and applications. [Online]. Available: https://rbcborealis.com/research-blogs/tutorial-9-sat-solvers-i-introduction-and-applications/

[45] The STP Project. STP • The Simple Theorem Prover. [Online]. Available: https://stp.github.io/

[46] Microsoft Corporation. Documentation for Online Z3 Guide — Online Z3 Guide. [Online]. Available: https://microsoft.github.io/z3guide/

[47] Group of Computer Architecture (AGRA). agra-uni-bremen/metaSMT. [Online]. Available: https://github.com/agra-uni-bremen/metaSMT

[48] Formal Security for Web Technologies. formalsec/smtml: A frontend for multiple SMT solvers in OCaml. [Online]. Available: https://github.com/formalsec/smtml

[49] N. Eén and S. Niklas. MiniSat Page. [Online]. Available: http://minisat.se/

[50] ADALogics. About us — ADA Logics. [Online]. Available: https://adalogics.com/about-us

[51] Frama-C. E-ACSL. [Online]. Available: https://frama-c.com/fc-plugins/e-acsl.html

[52] OWASP Foundation. (2023) OWASP Benchmark — OWASP Foundation. [Online]. Available: https://owasp.org/www-project-benchmark/